String matching algorithms are fundamental tools in computer science and are widely used in various applications such as text processing, data mining, information retrieval, and pattern recognition. These algorithms aim to locate occurrences of a pattern within a larger text or string.

The **string matching problem** : Given a text array, T [1…..n], of n character and a pattern array, P [1……m], of m characters. The problems are to find an integer s, called a valid shift where $0 \leq s < n-m$ and T [s+1……s+m] = P [1……m]. In other words, to find even if P in T, i.e., where P is a substring of T. The items of P and T are characters drawn from some finite alphabet such as {0, 1} or {A, B …..Z, a, b….. z}.

Brute-Force or Naive String Search algorithm

- searches for a string (also called pattern) within larger string.

It checks for character matches of pattern at each index of string.

If all characters of pattern match with string then search stops.

If not, it shifts to the next index of string for check.

It has worst case complexity of O(mn). Where m is length of pattern and n is length of string.

String

A	B	C	A	B	A	B		A	B	A	B	A	B	A	A	B	A	C
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

A	B	A	B	A	A	B	A
0	1	2	3	4	5	6	7

Pattern

S=0, start matching from the first location, once not matched shift the pattern matching from next place.

Now, S=1. Continue the process. In the above example, Pattern P occurs with shift ‘S’ =8 in ‘T’

0<=s<=n-m i.e 0<=s<=18-7

Algorithm

n=T.length

m=P.length

for s=0 to n-m

if P[1…m]==T[s+1…s+m]

print “pattern found with shift” s

Rabin Karp Algorithm

- It calculates a Hash Value for the pattern and subsequence of text to be compared.
- If hash(p) = hash(t_s) then compare the strings.
- To compute pattern:
  $t_{s+1} \leftarrow (d*(t_s - T[s+1]) * h) + T[s+m+1]$
  t_s = values of m-length substrings T[(s+1),(s+2)……,(s+m)] for s=0,1,2,….n-m
  m =window size, d=decimal value

If Hash of Subset of T and Hash of P matches then 2 conditions can be present. If strings are given then, A,B,…Z is considered as 1,2,3,….26. So if the string is ABC, then the hash value will be 1+2+3=6

- Both the character sets match ex., 564 from T and 564 as pattern match, the result is called **valid hit**
- Both the character sets don’t match, ex., 964 from T, 564 both hash values are 4 but characters don’t match, the result is called **spurious hit**
-

Ex.

T = 3456478964

P=564

Find out Hashing = Select any Random Prime No, ex 5

Now P mod q= 564 mod 5 =4

Find T subsets of length same as length of P, i.e 345,456,564…

Consider hashing of every subset i.e, 345 mod 5=0, 456 mod 5=1 etc

According to algorithm to compute the subset,

T = 3456478964, index starts from 1,2…n

P=564, P mod q= 564 mod 5 =4

T_0 = 345

345 mod 5 = 0 (hash of first T- subset)

T_1 = Using the formula above, considering d=10(if digits), =26(if alphabets), $h=d^{m-1}$

$(d*(t_s - T[s + 1]) * h) + T[s + m + 1]$

$= 10*(345- 10^{3-1} * T[0+1])+ T[0+3+1]$

$= 10*(345-100 * 3)+ 6$ (T[4] = 6)

$= 10*45 +6 =456$

456 mod 5 = 1

Algorithm

```
RABIN-KARP-MATCHER (T, P, d, q)
 Algorithm RABIN_KARP(T, P)
// T is text of length n
// P is pattern of length m

h ← power(d, m – 1) mod q
p ← 0
t0 ← 0

for i ← 1 to m do
   p ← (d*p + P[i]) mod q
   t0 ← ((d * t0) + T[i]) mod q
end

for s ← 0 to n – m do
   if p == t0 then
      if P[1…m] == T[s+1 … s + m] then
         print "Match found at shift", s
      end
   end
   if s < (n – m) then
      ts + 1 ← (d*(ts – T[s + 1]) * h ) + T[s + m + 1]
   end
end
```

d=alphabet size, q=prime number,

p_i ={a,b,c….,z}=d=26

Python program for Rabin Karp

```
d = 10
def search(pattern, text, q):
    m = len(pattern)
    n = len(text)
    p = 0
    t = 0
    h = 1
    i = 0
    j = 0
    for i in range(m-1):
        h = (h*d) % q
        # Calculate hash value for pattern and text
    for i in range(m):
        p = (d*p + ord(pattern[i])) % q
        t = (d*t + ord(text[i])) % q
# Find the match
    for i in range(n-m+1):
        if p == t:
            for j in range(m):
                if text[i+j] != pattern[j]:
                    break
            j += 1
            if j == m:
                print("Pattern is found at position: " + str(i+1))
        if i < n-m:
            t = (d*(t-ord(text[i])*h) + ord(text[i+m])) % q
            if t < 0:
                t = t+q
text = "ABCCDDAEFG"
pattern = "CDD"
q = 13
search(pattern, text, q)
```

What is Hashing?

- **Hashing** is a technique used for storing and retrieving information as quickly as possible. It is used to perform optimal searches.
- The time required to search an element in an array or linked list is either O(logn) or O(n) based on the list is sorted(binary search) or not (linear search). New data structure called Hashing is used to store and retrieve any entry with constant time O(1).

Components of Hashing
–Hashing has four key components:
1) Hash Table
2) Hash Functions
3) Collisions
4) Collision Resolution Techniques

Hash Table

Hash table or hash map is a data structure that stores the keys and their associated values
hash table is implemented using array. An element is accessed in O(1) time, if we know the key or its index

Example: To store the Student record in hash table, Student rollno is used as a key. If k is the student roll no. then that student record is stored at h(k) and not at k. It means a hash function h is used to calculate the index at which the element with key k will be stored. This process of mapping the keys to appropriate locations (or indices) in a hash table is called *hashing.*

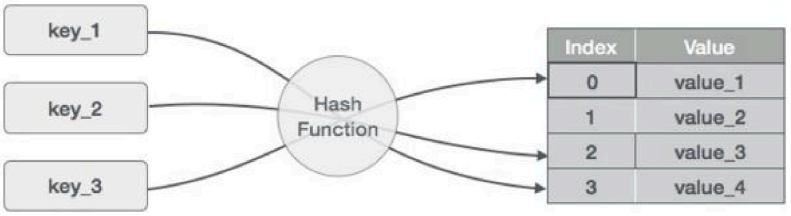
1. With an array, the element is stored whose key is k at a position k of the array. That means, given a key k, we find the element whose key is k by just looking in the kth position of the array. This is called **direct addressing.**
2. Use of hash tables. Hash table or hash map is a data structure that stores the keys and their associated values, and hash table uses a hash function to map keys to their associated values. The general convention is that a

hash table is used when the number of keys actually stored is small relative to the number of possible keys.

Hash Function

- **A hash function is a mathematical formula** which, when applied to a key, produces an integer which can be used as an index for the key in the hash table. A hash function **h** transforms a key into an index in a hash table T[0…m-1]: Where m is size of hash table.

- Given a collection of elements, a hash function that maps each item into a unique slot is referred to as a perfect hash function.



- when two or more keys map to the same memory location, a **collision** is said to occur. Goal is to create a hash function that minimizes the number of collisions, is easy to compute, and evenly distributes the elemems in the hash table.
-

How to Choose Hash Function?

The basic problems associated with the creation of hash tables are:
• An efficient hash function should be designed so that it distributes the index values of inserted objects uniformly across the table.
• An efficient collision resolution algorithm should be designed so that it computes an alternative index for a key whose hash index corresponds to a location previously inserted in the hash table.
• We must choose a hash function which can be calculated quickly, returns values within the range of locations in our table, and minimizes collisions.

- Have a high load factor for a given set of keys

HashTable ADT

The common operations for hash table arc:
• CreatHashTable: Creates a new hash table
• HashSearch: Searches the key in hash Lablc

• Hashinsert: Inserts a new key into hash table
• HashDelete: Deletes the key from hash table
• DeleteHashTable: Deletes the hash table

Types of Hash Functions:

There are various types of hash function which are used to place the data in a hash table,
1) Division Method
2)Mid square Method
3)Digit folding Method

1)Division Method

In this method the hash function is dependent upon the remainder of a division.
For example if the record 52, 68, 99, 84 is to be placed in a hash table and let us take the table size is 10.
Then:
Division Method h(key)= key % table_size
h(52) = 52 % 10 = 2
h(68) = 68 % 10 = 8
h(99) = 99 % 10 = 9
h(84) = 84 % 10 = 4

Hash Table

0	
1	
2	52
3	
4	84
5	
6	
7	
8	68
9	99

2)**Mid Square Method**
In this method firstly key is squared and then mid part of the result is taken as the index.
For example: consider that if we want to place a record of 3101 and the size of table is 1000. So, 3101*3101=9616201

i.e. h (3101) = 162 (middle 3 digit).

If the square value contains even number of digits the index is 0. If it contains odd value index is middle value. If the square value is a single digit, the value will be placed in that index only. Value 3 is stored at location 9.

k	k*k	Square	index h(k)
15	15 * 15	225	2
22	22 * 22	484	8
16	16 * 16	256	5
29	29 * 29	841	4
31	31 * 31	961	6
5	5 * 5	25	0
3	3 * 3	9	9

3)

Digit Folding Method

In this method, we partition the identifier k into several parts. All parts, except for the last one have the same length. We then add the parts together to obtain the hash address for k. There are two ways of carrying out this addition. In the first method, we shift all parts except for the last one, so that the least significant bit of each part lines up with the corresponding bit of the last part. We then add the parts together to obtain h(k). This method is known as **shift folding.**

The second method, known as **folding at the boundaries**, the key is folded at the partition boundaries, and digits falling into the same position are added together to obtain h(k). This is equivalent to reversing every other partition before adding.

Example 1: Suppose that k=12320324111220, and we partition it into parts that are 3 decimal digits long. The partitions are P1=123, P2=203, P3=241, P4=112 and P5=20

Shift Folding

$$h(k) = \sum_{i=1}^{5} P_i$$

= P1+P2+P3+P4+P5
= 123 + 203 + 241 + 112 + 20
= 699

Folding at the boundaries

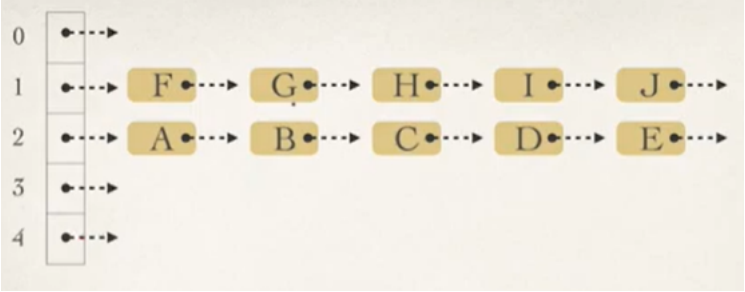
When folding at boundaries is used, we first reverse P2 and P4 to obtain 302 and 211 respectively. Next the five partitions are added to obtain

$$h(k) = \sum_{i=1}^{5} P_i$$

= P1+P2+P3+P4+P5
= 123 + 302 + 241 + 211 + 20
= 897

Example 2) For example: consider a record of 12465512 then it will be divided into parts. i.e. 124, 655, 12.
After dividing the parts combine these parts by adding it.
Shift folding H(key) =124+655+12 = 791
791 is the index to store the value 12465512
Folding at the boundaries H(key) =124+556+12 = 692 692 is the index to store the value 12465512

Load Factor: The load factor of a non-empty hash table is the number of items stored in the table divided by the size of the table. This is the decision parameter used when we want to rehash or expand the existing hash table entries. It helps in determining the efficiency of the hashing function (whether the hash function is distributing the keys uniformly or not).



Here buckets 0,3,4 have no elements so load is 0 for these buckets. So average of buckets=0+5+5+0+0=10
Load factor = 10/5 =2

Ex. Given a hash table T with 25 slots that stores 2000 elements the load factor for T is,
Load Factor = 2000/25=80

Collisions

Collisions occur when the hash function maps two different keys to the same location. Two records cannot be stored in the same location. Therefore, a method used to solve the problem of collision is required called collision resolution technique.